

# Responsive Web Design

## Implementation: Project Documentation

### 1. Design Choices

For this project, I adopted a **Mobile-First methodology**. This means the default styles in the CSS are written for the smallest screens, and media queries are used to progressively enhance the layout as the screen width increases.

- **Semantic HTML:** I used structural HTML5 tags like `<header>`, `<nav>`, `<main>`, `<article>`, `<aside>`, and `<footer>`. This improves accessibility for screen readers and boosts SEO, ensuring the document has meaningful structure rather than just a collection of `<div>` tags.
- **Fluid Typography:** Instead of relying entirely on media queries to change font sizes, I implemented the CSS `clamp()` function on the root body element and the hero heading. For example, `font-size: clamp(1rem, 0.95rem + 0.25vw, 1.25rem);` ensures the text smoothly scales between a minimum and maximum size based on the viewport width, providing a better reading experience without sudden jumps.
- **Relative Units:** I favored `rem` (root em) for spacing (margins/padding) and sizing to ensure accessibility. If a user changes their browser's default font size, the layout will scale proportionately. `vw` (viewport width) was used alongside `clamp()` for typography.

### 2. Layout Adjustments

The layout relies on modern CSS layout modules to handle responsiveness effortlessly:

- **Navigation:** On mobile devices, the navigation links stack in a column layout using CSS Flexbox. Once the screen reaches the tablet breakpoint (768px), a media query shifts the Flexbox direction to a row, placing the logo on the left and links on the right.
- **Content Grid:** The main content area (`.content-layout`) utilizes **CSS Grid**.
  - On mobile screens, it defaults to a single column (`grid-template-columns: 1fr;`), stacking the main article on top of the sidebar.
  - On screens 768px and wider, the grid adjusts to a two-column layout (`grid-template-columns: 2fr 1fr;`), giving the main article more space while pulling the aside panel to the right side.

### 3. Challenges & Testing

During the testing phase, I encountered a few specific challenges that required careful CSS adjustments:

- **Horizontal Scrolling Issues:** Initially, content was pushing past the edge of the viewport on narrow screens. I resolved this by applying a global CSS reset (`box-sizing: border-box;` and `margin: 0; padding: 0;`) and ensuring images were set to `max-width: 100%;` so they would never overflow their parent containers.

- **Touch Target Sizing:** On mobile, navigation links can be difficult to tap if they are clustered too closely. I increased the padding and gap values in the Flexbox navigation specifically for the mobile base styles to ensure they meet accessibility standards for touch targets.
- **Breakpoint Selection:** Choosing the right media query breakpoints can be tricky. Rather than targeting specific device models (like iPhone 14 or iPad Pro), I targeted layout breaks. I settled on 768px (typical tablet transition) and 1024px (desktop transition) because that is where the design naturally looked like it needed more structural complexity.